

CS & IT ENGINEERING



Operating System

Process Synchronization

Lecture -5

By- Vishvadeep Gothi sir



Recap of Previous Lecture



Doubt ?



Topic

Semaphore

Topics to be Covered



Topic

Questions on Semaphore

Topic

Bounded Buffer Problem

#Q. Consider a non-negative counting semaphore S . The operation $P(S)$ decrements S , and $V(S)$ increments S . During an execution, 43 $P(S)$ operations and 27 $V(S)$ operations are issued in some order. The largest initial value of S for which at least 5 $P(S)$ operation will remain blocked is 11?

43 $P(S) \Rightarrow 5$ should be blocked $\Rightarrow 43 - 5 = 38$ successful
27 $V(S) \Rightarrow$ all successful

$$x + 27 - 38 = 0$$

$$x = 11$$

[NAT]



#Q. Consider a non-negative counting semaphore S . The operation $P(S)$ decrements S , and $V(S)$ increments S . During an execution, 20 $P(S)$ operations and 12 $V(S)$ operations are issued in some order. The largest initial value of S for which at least one $P(S)$ operation will remain blocked is 7.?

[2016]

successful $P(S) = 20 - 1 = 19$

$$x + 12 - 19 = 0$$

$$x = 7$$

[NAT]



Ans = 2

- #Q. A shared variable x , initialized to zero, is operated on by four concurrent processes W, X, Y, Z as follows. Each of the process W and X reads x from memory, increments by one, stores it to memory and then terminates. Each of the processes Y and Z reads x from memory, decrements by two, stores it to memory and then terminates. Each processes before reading x invokes the P operation (i.e., wait) on a counting semaphore S and invokes the V operation (i.e., signal) on the semaphore S after storing x to memory. Semaphore S is initialized to two. What is the maximum possible value of x after all processes complete execution?

[2013]

$$x = 2$$

W
 $P(S)$
 $x = x + 1$
 $V(S)$

X
 $P(S)$
 $x = x + 1$
 $V(S)$

Y
 $P(S)$
 $x = x - 2$
 $V(S)$

Z
 $P(S)$
 $x = x - 2$
 $V(S)$

$S = 2 \Rightarrow$ no mutual exclusion $\Rightarrow$ Race condition

$x = 0$

$\Downarrow$
 max value of x will have cumulative impact of process W and X.

$$x = 0 + 1 + 1 = 2$$

Ques) In prev. questⁿ, min. possible value of $x = \underline{-4}$?

$$0 - 2 - 2 = -4$$

Ques) In prev. ques, no. of distinct value of x possible 7?

$$0 + 1 + 1 - 2 - 2$$

$$\left. \begin{array}{r} 1 \\ -2 \\ 2 \\ -4 \\ -1 \\ 0 \\ -3 \end{array} \right\} \text{race condition}$$

Ques) In prev. questⁿ if semaphore S is initialized by 1,
then

$$\text{max value } x = \underline{-2}$$

$$\text{min value } x = \underline{-2}$$

$$\text{all distinct possible values of } x = \underline{1}$$

↓
mutual exclusion

↓
no race condition

#Q. A shared variable x , initialized to zero, is operated on by four concurrent processes W, X, Y, Z as follows. Each of the process W and X reads x from memory, increments by 2, stores it to memory and then terminates. Each of the processes Y and Z reads x from memory, decrements by 3, stores it to memory and then terminates. Each processes before reading x invokes the P operation (i.e., wait) on a counting semaphore S and invokes the V operation (i.e., signal) on the semaphore S after storing x to memory. Semaphore S is initialized to two. What are the total distinct possible values of x after all processes complete execution?

no mutual exclusion

↓
race condition

W	X	Y	Z
$+2$	$+2$	-3	-3

2
-3
4
-6
-1
1
-4
-2

Ans = 8

[MCQ]



#Q. Consider the two functions incr and decr shown below.

```
incr() {                                decr(){
    wait(s);                            wait(s);
    X = X+1;                            X = X-1;
    signal(s);                          signal(s);
}
```

There are 5 threads each invoking incr once, and 3 threads each invoking decr once, on the same shared variable X. The initial value of X is 10.

Suppose there are two implementations of the semaphore s, as follows:

I1: s is a binary semaphore initialized to 1. $\Rightarrow$ mutual exclusion $= 10 + 5 - 3 = 12$

$10 - 3 = 7$ $\Leftarrow$ I2: s is a counting semaphore initialized to 2. $\Rightarrow$ Race condition (no mutual exclusion)

Let V1, V2 be the values of X at the end of execution of all the threads with implementations I1, I2, respectively.

Which one of the following choices corresponds to the minimum possible values of V1, V2, respectively? **[2023]**



15, 7



7, 7



✓ 12, 7



12, 8

Busy waiting :-

A process runs on CPU but still waits to progress further.

`while(True);` $\Rightarrow$ busy waiting

`wait(s)`

`{ while(s <= 0);`

`s--;`

`}`



Process eats CPU's time unnecessarily.

best solution $\Rightarrow$ if CS is occupied then block the process until CS is free.



```
wait(Semaphore s){
    s=s-1;
    if (s<0) {
        // add process to queue
        block();
    }
}
```

P1

PZ

P3

```

signal(Semaphore s){
    s=s+1;
    if (s<=0) {
        // remove
        process p from queue
        wakeup(p);
    }
}

```

P1 in

wait(s)
CS
signal(s)

P1 in CS

$$S = \cancel{1} \cancel{0} \cancel{1}$$
$$\quad \quad \quad \cancel{2}$$
$$\quad \quad \quad \cancel{1} \cancel{0}$$
$$\quad \quad \quad \quad 1$$


by Default $\Rightarrow$ busy waiting in solutions using semaphore.



Topic : Starvation for Critical Section



In CPU scheduling $\Rightarrow$ Starvation = A process not getting CPU

In process synch. $\Rightarrow$ Starvation = A process getting CPU
but not getting critical section

#Q. Given below is a program which when executed spawns two concurrent processes:

Semaphore X: ~~=0~~; ~~1~~ ~~0~~ ~~1~~ 0

/* Process now forks into concurrent processes P1 & P2 */

P1 : repeat forever

V(X);

Compute;

P(X);

P2: repeat forever

P(X);

Compute;

V(X);

Consider the following statements about processes P1 and P2:

- ✓ I. It is possible for process P1 to starve. $\Rightarrow$ True
- ✓ II. It is possible for process P2 to starve. $\Rightarrow$ True

Solution 1

✓ starvation

Boolean lock=~~false~~; *True*

P0
 while(true)
 {
 while(lock);
 lock=true;
 → //CS
 lock=false;
 RS;
 }

P1

while(true)
 {
 → while(lock);
 lock=true;
 //CS
 lock=false;
 RS;
 }

Solution 2

P_0, P_1 both can starve
 $\Rightarrow$ solⁿ suffers from starvation

```
int turn=0;
```

```
while(true)  $P_0$ 
{
    while(turn!=0);
    CS
    turn=1;
    RS;
}
```

P_1

```
while(true)
{
    while(turn!=1);
    CS
    turn=0;
    RS;
}
```

Peterson's Solution

No starvation

```
Boolean Flag[2]={False, False};
int turn;
```

```
while(true) {
    Flag[0]=true;
    turn=1;
    while(Flag[1] && turn==1);
        CS
    Flag[0]=False;
    RS;
}
```

```
while(true){
    Flag[1]=true;
    turn=0;
    while(Flag[0] && turn==0);
        CS
    Flag[1]=False;
    RS;
}
```



Topic : TestAndSet()



Starvation = —?

```
Boolean Lock=False;
```

```
boolean TestAndSet(Boolean *trg){
```

```
boolean rv = *trg;
```

```
*trg = True;
```

```
Return rv;
```

```
}
```

```
while(true)
```

```
{
```

```
while(TestAndSet(&Lock));
```

```
CS
```

```
Lock=False;
```

```
}
```



Topic : Swap()



starvation ?

Boolean Key; //Local

Boolean Lock=False; //Shared

void Swap(Boolean *a, Boolean *b)

```
{  
    boolean temp = *a;  
    *a=*b;  
    *b=temp;  
}
```

```
while(true){  
    Key = True;  
    while (key==True)  
        Swap(&Lock, &Key);  
    CS  
    Lock=False;  
}
```



Topic : Classical Problems of Synchronization



- Producer Consumer problem (bounded buffer problem)
- Reader - writer "
- Dining philosopher "





Topic : Bounded Buffer Problem

shared b/w all producers and consumers

Bounded buffer with capacity N

Multiple
producers



Multiple
consumers



Topic : Bounded Buffer Problem

- Known as producer-consumer problem also
- Buffer is the shared resource between producers and consumers





Topic : Bounded Buffer Problem : Solution

- Producers must block if the buffer is full
- Consumers must block if the buffer is empty





Topic : Bounded Buffer Problem : Solution

- **Variables:**

- Mutex: Binary Semaphore to take lock on buffer (Mutual Exclusion)
- Full: Counting Semaphore to denote the number of occupied slots in buffer
- Empty: Counting Semaphore to denote the number of empty slots in buffer

Initialization:- (Initially buffer should be empty)

$\text{mutex} = 1$

$\text{Full} = 0$

$\text{Empty} = N$

when buffer becomes full.

$\text{Full} = N$

$\text{Empty} = 0$



Topic : Producer() & Consumer

Producer

wait (Empty)
wait (mutex)
// add item on buffer
signal (mutex)
signal (Full)

Consumer

wait (Full)
wait (mutex)
// remove item from buffer
signal (mutex)
signal (Empty)

Deadlock:-

2 processes are in situation where both can never proceed further.



Topic : Producer() & Consumer

Producer

$\rightarrow$ wait(mutex) $\leftarrow$
 $\rightarrow$ wait(empty) $\leftarrow$
// add item on buffer
signal(mutex)
signal(Full)

when buffer is full $\Rightarrow$ Deadlock

mutex = ~~1~~ 0
Full = ~~n~~ n-1
empty = 0

Consumer

$\rightarrow$ wait(Full)
 $\rightarrow$ wait(mutex)
// remove item from buffer
signal(mutex)
signal(empty)



Topic : Producer() & Consumer

Producer

wait(Empty)
→ wait(mutex)
// add item on buffer
signal(mutex)
signal(Full)

Consumer

→ wait(mutex)
wait(Full)
// remove item from buffer
signal(mutex)
signal(Empty)

when buffer is empty $\Rightarrow$ Deadlock

mutex = ~~1~~ 0
Full = 0
Empty = ~~n~~ n-1



2 mins Summary

Topic

Questions on Semaphore

Topic

Bounded Buffer Problem



Happy Learning

THANK - YOU

